

Kubernetes Cluster Upgrade (kubeadm - On-Prem)

What is kubeadm Upgrade?

- 👉 In self-managed (on-prem) Kubernetes, you upgrade clusters manually using kubeadm.
- 💡 Unlike managed services (EKS), you control everything : Control plane upgrade & Worker node upgrade

-----🚀🚀master node🚀🚀-----
🚫 Step 1: Drain master node

```
kubectl cordon master-1                                # P
kubectl drain master-1 --ignore-daemonsets --delete-emptydir-data # 🏆
```

📦 Step 2: Upgrade kubeadm

```
apt-mark unhold kubeadm                                # Unhold kubeadm so you can u
apt-get update                                           #Update package list and inst
apt-get install -y kubeadm=1.24.17-00
apt-mark hold kubeadm                                   # Lock version (avoid auto u
```

🚀 Step 3: Upgrade control plane

```
kubeadm upgrade plan                                # Check upgrade plan
kubeadm upgrade apply v1.24.17                       # Apply upgrade
```

🔄 This upgrades control plane components lik: kube-apiserve

⚙️ Step 4: Upgrade node components (kubelet and kubectl)

```
apt-mark unhold kubelet kubectl
apt-get install -y kubelet=1.24.17-00 kubectl=1.24.17-00
apt-mark hold kubelet kubectl
```

🔄 Step 5: Restart services

```
systemctl daemon-reexec                                # Reload systemd to pick up changes (sa
systemctl restart kubelet                              # Restart kubelet to apply the upgrade
```

✅ Step 6: Uncordon the master node (Enable scheduling)

```
kubectl uncordon master-1
```

```
# Uncordon the master node to allow sch
```

-----🧑🧑 Worker Node🧑🧑-----

```
### 🧑 B. Worker Node Upgrade
```

```
# 🚫 Step 1: Drain worker node
```

```
kubectl cordon worker-1
```

```
kubectl drain worker-1 --ignore-daemonsets --delete-emptydir-data
```

```
# 📦 Step 2: Upgrade kubeadm
```

```
apt-mark unhold kubeadm
```

```
apt-get update
```

```
apt-get install -y kubeadm=1.24.17-00
```

```
apt-mark hold kubeadm
```

```
# 🔄 Step 3: Apply worker node upgrade (Update kubelet config)
```

```
kubeadm upgrade node
```

```
# This updates the local kubelet con
```

```
# ⚙️ Step 4: Upgrade kubelet & kubectl
```

```
apt-mark unhold kubelet kubectl
```

```
apt-get install -y kubelet=1.24.17-00 kubectl=1.24.17-00 # Install th
```

```
apt-mark hold kubelet kubectl
```

```
# 🔄 Step 5: Restart services
```

```
systemctl daemon-reexec
```

```
#Reload systemd and resta
```

```
systemctl restart kubelet
```

```
# ✅ Step 6: Uncordon the Worker Node ( Resume scheduling )
```

```
kubectl uncordon worker-1
```

```
# pods to be scheduled
```

💡 Quick Flow (Easy to Remember)

upgrade_flow Interview shortcut:

- Cordon

- Drain

- Upgrade kubeadm

- Upgrade control plane / node

- Upgrade kubelet & kubectl

🚫 Stop scheduling

🧹 Evict pods

📦 Tool upgrade

🚀 Apply upgrade

⚙️ Match versions

- Restart services
- Uncordon

- #  Apply changes
- #  Resume scheduling

Kubernetes Upgrade Steps (Master vs Worker)

 Step	 Master Node	 Worker Node	 Explanation
 Cordon & Drain			 Safely evict pods before upgrade
 Upgrade kubeadm			 Update kubeadm binary
 kubeadm upgrade apply			 Upgrades control plane components
 kubeadm upgrade node			 Updates node configuration
 Upgrade kubelet & kubectl			 Update node agents & CLI
 Restart services			 Restart kubelet to apply changes
 Uncordon			 Allow scheduling again

How to Verify Cluster Health After Upgrade?

Basic Checks

-  After any upgrade (EKS or kubeadm), verification is CRITICAL

Basic Health Checks (CLI)

```
kubectl get nodes          # Check Nodes          ## Ensure: All nodes are Ready &
kubectl get pods -A        #  Ensure: All pods are Running / Completed & No C
kubectl describe nodes     #  Look for: Resource pressure (CPU, Memory) & Net
```

Monitoring Checks

- CloudWatch Logs 
- Prometheus Metrics 
- Grafana Dashboards & Alerts 

Version Compatibility Rule

- 👉 Kubernetes supports:
 - Control Plane ↔ Kubelet = ± 1 minor version
- ⚠️ If kubelet is older:
 - Node may become unschedulable

Upgrade Worker Nodes (Zero Downtime)

Blue-Green Style Approach

- 1 Create new node group (updated version)
- 2 Cordon old nodes:

```
kubect1 cordon <node>
```

- 3 Drain pods safely:

```
kubect1 drain <node> --ignore-daemonsets --delete-emptydir-data
```

- 4 Move workloads to new nodes
- 5 Delete old node group (Zero downtime achieved)

How to Ensure Zero Downtime?

Best Practices :

- Use Rolling Updates 
- Configure PodDisruptionBudgets (PDB) 
- Use Readiness & Liveness Probes  (Only healthy pods receive traffic)
- Maintain multiple replicas 
- Monitor application health  (Detect issues early)

Upgrade Methods

☁ Managed Kubernetes (EKS, GKE, AKS)

- Control plane managed by cloud provider
- Node groups upgraded manually or via automation
- Tools:
 - UI (AWS Console)
 - CLI (eksctl , gcloud , az)
 - Terraform

🧱 Self-Managed Kubernetes

- Manual upgrade required:
- kubeadm
- kubelet
- kubectl
- Add-ons
- Config files

🔄 Upgrade Responsibility: EKS vs kubeadm

 Component	 Amazon Elastic Kubernetes Service	 kubeadm (Self-Managed)	 Explanation
 Control Plane	✅ AWS handles (version upgrades , HA , patches)	❌ You handle (kube-apiserver , etcd , scheduler , controller)	Fully managed vs full responsibility
 Worker Nodes	⚠️ You manage (node groups / AMI upgrades)	❌ You manage fully	In EKS, AWS provides tooling but you trigger upgrades
 Add-ons	⚠️ Manual / AWS-managed add-ons (VPC CNI , CoreDNS , kube-proxy)	❌ Manual	EKS offers managed add-ons but still needs planning (Easier in EKS)
 etcd	✅ AWS manages (No access)	❌ You manage	Hidden in EKS, visible in kubeadm (full control)
 Certificates	✅ AWS handles control plane certs	❌ You rotate/manage (kubeadm requires cert renewal)	Security responsibility

Smoke Testing

- 👉 Quickly verify:
 - App endpoints working 🌐
 - APIs responding ⚡
 - No errors in logs ❌

kubeadm Cluster Upgrade – Interview Answer

◆ How to Explain in an Interview

- In our on-premises Kubernetes cluster managed using kubeadm, I upgrade master and worker nodes separately to ensure high availability and minimal downtime.

✅ ◆ Master Node Upgrade Explanation

- I start by cordoning and draining the master node using `kubect1` so that no new pods are scheduled and existing ones are safely evicted .
- Then I upgrade the kubeadm binary to the target version using the package manager (like `apt`).
- After that, I verify the upgrade plan using:

```
kubeadm upgrade plan
```

- Then I apply the upgrade:

```
kubeadm upgrade apply <version>
```

- 👉 This upgrades control plane components like:
 - API Server
 - Controller Manager
 - Scheduler
- Next, I upgrade:

- `kubelet`
- `kubectl`
- Then I restart the `kubelet` service to apply changes.
- Finally, I uncordon the node so it can start `scheduling pods` again.
- 👉 I repeat this process for all master nodes in a `multi-master setup` .



◆ Worker Node Upgrade Explanation

- For worker nodes, the process is similar but simpler.
- I start by cordoning and draining the node.
- Then I upgrade the `kubeadm` binary.
- Since worker nodes don't run control plane components, I run:

```
kubeadm upgrade node
```

- 👉 This updates `kubelet` configuration.
- Next, I upgrade:
 - `kubelet`
 - `kubectl`
- Then restart `kubelet` and uncordon the node.
- 👉 I upgrade one node at a time to ensure workloads continue running smoothly.



Final Summary Line

- By upgrading nodes one by one and continuously monitoring cluster health, I ensure a stable, zero-downtime upgrade process using `kubeadm`.



Key Highlights

- 🔄 Sequential upgrade (`Master` → `Worker`)
- 🚫 Cordon & Drain before upgrade
- 🔄 Rolling upgrade approach
- 🔍 Continuous monitoring

- ⚡ Zero downtime achieved with proper planning
-

Summary

- This approach ensures Kubernetes cluster upgrades are performed safely with minimal risk ,maintaining application availability and cluster stability throughout the process.
- 👉 Smooth upgrade = Reliable production 🚀